

L'uso di un sistema di controllo di versione (VCS) non è solo per programmatori, ma per chiunque voglia tenere traccia delle modifiche ai propri file nel tempo, recuperare versioni precedenti e collaborare senza il rischio di sovrascrivere il lavoro altrui.

1. L'evoluzione dei Sistemi di Controllo Versione

I sistemi per gestire le versioni dei file si sono evoluti in tre macro-categorie:

- **Sistemi Locali:** Memorizzano le diverse versioni dei file in un semplice database sul computer. Se da un lato evitano gli errori di copiatura manuale, dall'altro sono vulnerabili: se si rompe il disco rigido e non ci sono backup, il lavoro va perso.
- **Sistemi Centralizzati (CVCS):** Utilizzano un server centrale che contiene tutti i file; gli utenti si connettono per prelevare ciò che serve. Questo agevola moltissimo il lavoro di squadra e l'amministrazione dei permessi, ma crea un grave punto debole (single point of failure): se il server va offline, nessuno può salvare o collaborare.
- **Sistemi Distribuiti (DVCS, es. Git):** Risolvono il problema del server centrale. Ogni utente che scarica il progetto ne clona l'intero database, compresa tutta la cronologia. In questo modo, ogni computer possiede un backup completo; se il server centrale si guasta, qualsiasi copia locale può essere usata per ripristinarlo.

2. Perché Git è diverso dagli altri

Git si distingue per due caratteristiche tecniche rivoluzionarie:

- **Istantanee, non differenze:** Molti sistemi tradizionali salvano solo le *differenze* (delta) tra un file e la sua versione precedente. Git, invece, scatta una "fotografia" (un'istantanea) dell'intero progetto in quel preciso momento. Se un file non è cambiato, Git non lo copia di nuovo, ma crea un collegamento alla versione già salvata. Questo rende quasi tutte le operazioni locali ed estremamente veloci.
- **Integrità assoluta (Checksum):** Tutto in Git viene verificato tramite un algoritmo chiamato SHA-1 hash (una stringa esadecimale di 40 caratteri) prima di essere memorizzato. Git non salva i file in base al loro nome, ma in base all'hash del loro contenuto. Di conseguenza, è letteralmente impossibile modificare un file, perderlo per un errore di rete o corromperlo senza che Git se ne accorga.

3. La Regola d'Oro di Git: Le 3 Aree e i 3 Stati

Per padroneggiare Git, devi capire che i tuoi file si sposteranno sempre tra tre ambienti principali, assumendo tre stati diversi:

- **Working Directory (Stato: *Modified*):** È la cartella vera e propria sul tuo computer in cui modifichi o crei i file.
- **Staging Area (Stato: *Staged*):** È un'area di parcheggio (un buffer) dove selezioni e prepari esattamente i file che vuoi includere nel prossimo salvataggio ufficiale, evitando così di includere errori o modifiche a metà.
- **Git Directory (Stato: *Committed*):** È il cuore del sistema (la cartella nascosta `.git`). Quando esegui un commit, i file passano qui e vengono archiviati in modo permanente e sicuro nel database.

4. Branching e Merging: Sviluppare in parallelo

Fare *branching* in Git significa creare percorsi di sviluppo paralleli all'interno dello stesso progetto senza dividerlo. A differenza di altri sistemi in cui questa operazione è lenta, su Git creare e cambiare branch è praticamente istantaneo.

Una tipica struttura organizzativa prevede:

- **Branch *master* (o *main*):** Contiene unicamente codice testato, assolutamente stabile e pronto per il rilascio in produzione.
- **Branch *develop*:** Il "cantierino" dove si uniscono e testano i vari aggiornamenti prima di portarli sul master.
- **Topic Branch:** Rami provvisori, usati per sviluppare singole funzionalità o sperimentare idee. Se l'esperimento fallisce, il branch si cancella; se funziona, il lavoro viene unito (*merge*) al resto del progetto in modo sicuro.

5. Prontuario dei Comandi Essenziali

Ecco i passaggi operativi fondamentali per usare Git tutti i giorni:

1. Configurazione Iniziale (Da fare solo la prima volta sul PC):

- Imposta il tuo nome e la tua email (informazioni necessarie per firmare i commit): `git config --global user.name "Nome Cognome" git config --global user.email nome@example.com`

2. Creare o Scaricare un Progetto:

- Se parti da zero con una tua cartella: `git init` (crea la cartella nascosta `.git` per il versionamento).
- Se vuoi scaricare un progetto esistente dal web: `git clone <url-del-repository>`.

3. Il Ciclo di Lavoro Standard:

- Aggiungi le modifiche all'area di staging: `git add ..`
- Salva ufficialmente l'istantanea: `git commit -m "Messaggio che descrive le modifiche"`.

4. Pubblicare su Internet (es. GitHub):

- Collega la cartella locale al server remoto: `git remote add origin <url-del-repository>`.
- Invia il codice online: `git push origin master (o main)`.

Per spostare i file dalla tua area di lavoro fino al salvataggio definitivo, utilizzerai un set di comandi ricorrenti.

- **git status:** Viene utilizzato per controllare lo stato attuale del repository. Mostra in quale branch ti trovi e se ci sono file modificati, aggiunti o non tracciati.
- **git add <file>:** Inizia a tracciare i nuovi file o aggiunge le modifiche dei file esistenti all'area di staging. I file in staging sono pronti per essere confermati nel commit.
- **git commit -m "messaggio":** Salva in modo permanente i file presenti nell'area di staging nella cronologia. È buona pratica includere sempre un messaggio per spiegare le modifiche apportate.
- **git diff:** Mostra cosa è cambiato nei file, evidenziando le righe aggiunte e rimosse. Mostra solo le modifiche non ancora aggiunte allo staging. Per controllare le modifiche già preparate per il commit, devi usare `git diff --staged`.
- **git log:** Mostra la cronologia delle modifiche, includendo gli autori, le date e i messaggi dei commit. Un'opzione molto utile è `git log --oneline`, che comprime ogni commit in una sola riga.
- **git rm <file>:** Rimuove un file sia dall'area di staging che dal repository. Se vuoi smettere di tracciare un file ma vuoi mantenerlo fisicamente sul tuo computer, devi usare `git rm --cached <file>`.

2. L'Insidia dello Staging

Un concetto fondamentale da capire è che Git ragiona per "stati" precisi nel tempo.

- Se modifichi un file e fai `git add`, quel file entra nello staging.
- Se lo modifichi di nuovo prima di fare il commit, Git considererà il file in due stati contemporaneamente: la prima modifica pronta per il commit (*staged*) e la nuova modifica ancora fuori (*unstaged*).

- La versione che finirà nel commit è solo quella in staging. Per includere le ultime modifiche, devi usare di nuovo git add.

3. Come Rimediare agli Errori (Undoing)

Tutti sbagliano, ma Git offre reti di salvataggio (da usare con cautela, poiché alcune fanno perdere dati definitivamente).

Errore	Comando Risolutivo	Effetto
Vuoi buttare via le modifiche a un file	git checkout <file>	Ripristina il file all'ultimo commit. Attenzione: le tue modifiche andranno perse per sempre.
Hai fatto git add per sbaglio	git reset HEAD <file>	Toglie il file dallo staging. Le tue modifiche rimangono intatte sul PC.
Hai scordato un file nell'ultimo commit	git add <file> seguito da git commit --amend	Apri l'editor per unire il nuovo file al commit precedente . Non usarlo se hai già inviato il codice online!

4. Branching e Merging (La "Killer Feature")

Git non salva differenze riga per riga, ma scatta una vera e propria fotografia (snapshot) di tutto il repository a ogni commit.

- **I Branch:** In Git, un branch non è altro che un puntatore leggero e mobile a uno di questi commit. Il branch master non ha nulla di speciale tecnicamente, è semplicemente il nome predefinito creato all'inizio.
- **Spostarsi (HEAD):** Git sa su quale branch ti trovi grazie a un puntatore speciale chiamato HEAD. Quando crei un branch con git branch <nome>, non ci finisci dentro in automatico. Devi usare git checkout <nome> per spostare l'HEAD su quel branch e iniziare a lavorarci.
- **Unire (Merging):** Per fondere il lavoro di due branch si usa git merge. Se il lavoro è andato dritto in modo lineare, Git fa un semplice **Fast-forward** (sposta avanti il puntatore). Se le storie dei due branch sono divergenti, Git fa un **Three-way merge**, creando un nuovo "commit di merge" che unisce i rami .

5. Gestione dei Conflitti e Lavoro su Server

Quando due branch modificano la stessa parte di un file in modo diverso, il merge automatico fallisce e si crea un conflitto.

- **Risolvere:** Devi usare git status per trovare i file in conflitto, aprirli, decidere manualmente quale versione tenere e infine fare git add e git commit per completare il merge.
- **Repository Bare:** Per collaborare in rete, i server ospitano repository "bare", ovvero file .git puri senza nessuna cartella di lavoro visibile associata. Si configurano solitamente tramite protocolli sicuri come SSH o HTTP .